

```
In [2]: %pip install opencv-python numpy matplotlib
```

```

Defaulting to user installation because normal site-packages is not writeable
Collecting opencv-python
  Downloading opencv_python-5.0.0.93-cp37-abi3-manylinux_2_28_x86_64.whl.metadata (19 kB)
Collecting numpy
  Using cached numpy-2.2.6-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (62 kB)
Collecting matplotlib
  Using cached matplotlib-3.10.9-cp310-cp310-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (52 kB)
Collecting contourpy>=1.0.1 (from matplotlib)
  Using cached contourpy-1.3.2-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (5.5 kB)
Collecting cycler>=0.10 (from matplotlib)
  Using cached cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.22.0 (from matplotlib)
  Downloading fonttools-4.65.0-cp310-cp310-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (125 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib)
  Downloading kiwisolver-1.5.1-cp310-cp310-manylinux_2_12_x86_64.manylinux2010_x86_64.whl.metadata (5.2 kB)
Requirement already satisfied: packaging>=20.0 in /home/Nandini/.local/lib/python3.10/site-packages (from matplotlib) (26.3)
Collecting pillow>=8 (from matplotlib)
  Downloading pillow-12.3.0-cp310-cp310-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (9.1 kB)
Collecting pyparsing>=3 (from matplotlib)
  Using cached pyparsing-3.3.2-py3-none-any.whl.metadata (5.8 kB)
Requirement already satisfied: python-dateutil>=2.7 in /home/Nandini/.local/lib/python3.10/site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /home/Nandini/.local/lib/python3.10/site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Downloading opencv_python-5.0.0.93-cp37-abi3-manylinux_2_28_x86_64.whl (73.8 MB)
_____ 73.8/73.8 MB 346.5 kB/s eta 0:00:00
0000:0100:05
Using cached numpy-2.2.6-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (16.8 MB)
Using cached matplotlib-3.10.9-cp310-cp310-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (8.8 MB)
Using cached contourpy-1.3.2-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (325 kB)
Using cached cycler-0.12.1-py3-none-any.whl (8.3 kB)
Downloading fonttools-4.65.0-cp310-cp310-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (5.3 MB)
_____ 5.3/5.3 MB 249.1 kB/s eta 0:00:00
a 0:00:02
Downloading kiwisolver-1.5.1-cp310-cp310-manylinux_2_12_x86_64.manylinux2010_x86_64.whl (1.6 MB)
_____ 1.6/1.6 MB 79.3 kB/s eta 0:00:00a
0:00:05
Downloading pillow-12.3.0-cp310-cp310-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (6.9 MB)
_____ 6.9/6.9 MB 239.6 kB/s eta 0:00:00
00:0100:02
Using cached pyparsing-3.3.2-py3-none-any.whl (122 kB)

```

Installing collected packages: pyparsing, pillow, numpy, kiwisolver, fonttools, cyclcr, opencv-python, contourpy, matplotlib

9/9 [matplotlib]9 [matplotlib]on]
Successfully installed contourpy-1.3.2 cyclcr-0.12.1 fonttools-4.65.0 kiwisolver-1.5.1 matplotlib-3.10.9 numpy-2.2.6 opencv-python-5.0.0.93 pillow-12.3.0 pyparsing-3.3.2

[notice] A new release of pip is available: 25.1.1 -> 26.2.1

[notice] To update, run: `pip install --upgrade pip`

Note: you may need to restart the kernel to use updated packages.

```
In [5]: import cv2
import numpy as np
import matplotlib.pyplot as plt

print("Libraries imported successfully!")
```

Libraries imported successfully!

```
In [6]: video_path = "video (Copy).mp4"

cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    print("Error: Could not open video.")
else:
    print("Video opened successfully!")

    width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    fps = cap.get(cv2.CAP_PROP_FPS)

    print("Width:", width)
    print("Height:", height)
    print("FPS:", fps)

cap.release()
```

Video opened successfully!

Width: 576

Height: 1024

FPS: 25.0

```
In [8]: cap = cv2.VideoCapture(video_path)
```

```
In [9]: print(video_path)
```

video (Copy).mp4

```
In [10]: import cv2

cap = cv2.VideoCapture(video_path)

print("Video opened:", cap.isOpened())

ret, frame = cap.read()
```

```
print("Frame read:", ret)
print("Frame:", frame.shape if ret else None)

cap.release()
```

Video opened: True
 Frame read: True
 Frame: (1024, 576, 3)

In [20]: `video_path = "video.mp4"`

```
In [22]: import cv2

cap = cv2.VideoCapture(video_path)

print("Opened:", cap.isOpened())
print("FPS:", cap.get(cv2.CAP_PROP_FPS))
print("Total frames:", int(cap.get(cv2.CAP_PROP_FRAME_COUNT)))

cap.release()
```

Opened: True
 FPS: 24.0
 Total frames: 240

```
In [23]: cap = cv2.VideoCapture(video_path)

ret, frame1 = cap.read()
ret2, frame2 = cap.read()

if ret and ret2:
    print("Two consecutive frames successfully read.")

    gray1 = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
    gray2 = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

    print("Frames converted to grayscale.")
else:
    print("Could not read frames.")

cap.release()
```

Two consecutive frames successfully read.
 Frames converted to grayscale.

```
In [25]: import cv2
import numpy as np

# First grayscale frame mein good features detect karo
p0 = cv2.goodFeaturesToTrack(
    gray1,
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)
```

```
print("Features detected:", len(p0) if p0 is not None else 0)
```

Features detected: 85

```
In [28]: # Lucas-Kanade Sparse Optical Flow

next_points, status, error = cv2.calcOpticalFlowPyrLK(
    gray1,
    gray2,
    p0,
    None
)

# Successfully tracked points
good_old = p0[status == 1]
good_new = next_points[status == 1]

print("Original points:", len(p0))
print("Tracked points:", len(good_new))
```

Original points: 85

Tracked points: 85

```
In [30]: import cv2
import numpy as np
import matplotlib.pyplot as plt

video_path = "video.mp4"

cap = cv2.VideoCapture(video_path)

ret, old_frame = cap.read()

if not ret:
    print("Error: Cannot read video.")
else:
    old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

    p0 = cv2.goodFeaturesToTrack(
        old_gray,
        mask=None,
        maxCorners=100,
        qualityLevel=0.3,
        minDistance=7,
        blockSize=7
    )

    lk_params = dict(
        winSize=(15, 15),
        maxLevel=2,
        criteria=(
            cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT,
            10,
            0.03
        )
    )
```

```

    print("Lucas-Kanade Optical Flow initialized.")

cap.release()

```

Lucas-Kanade Optical Flow initialized.

```

In [34]: import cv2
import numpy as np
import matplotlib.pyplot as plt

video_path = "video.mp4"

# Open video again
cap = cv2.VideoCapture(video_path)

# Read first frame
ret, old_frame = cap.read()

if not ret:
    print("Error: Cannot read video.")
else:
    old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

    # Detect feature points
    p0 = cv2.goodFeaturesToTrack(
        old_gray,
        maxCorners=100,
        qualityLevel=0.3,
        minDistance=7,
        blockSize=7
    )

    # Lucas-Kanade parameters
    lk_params = dict(
        winSize=(15, 15),
        maxLevel=2,
        criteria=(
            cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT,
            10,
            0.03
        )
    )

    # Read second frame
    ret, frame = cap.read()

    if ret:
        frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        # Calculate optical flow
        p1, status, error = cv2.calcOpticalFlowPyrLK(
            old_gray,
            frame_gray,
            p0,
            None,

```

```

        **lk_params
    )

    if p1 is not None:
        good_new = p1[status == 1]
        good_old = p0[status == 1]

        output = frame.copy()

        # Draw motion arrows
        for new, old in zip(good_new, good_old):
            x_new, y_new = new.ravel()
            x_old, y_old = old.ravel()

            x_new, y_new = int(x_new), int(y_new)
            x_old, y_old = int(x_old), int(y_old)

            cv2.arrowedLine(
                output,
                (x_old, y_old),
                (x_new, y_new),
                (0, 255, 0),
                2,
                tipLength=0.3
            )

            cv2.circle(
                output,
                (x_new, y_new),
                4,
                (0, 0, 255),
                -1
            )

        # Display result
        output_rgb = cv2.cvtColor(output, cv2.COLOR_BGR2RGB)

        plt.figure(figsize=(12, 7))
        plt.imshow(output_rgb)
        plt.title("Lucas-Kanade Sparse Optical Flow")
        plt.axis("off")
        plt.show()

        print("Lucas-Kanade Optical Flow completed.")
        print("Tracked points:", len(good_new))
    else:
        print("No points could be tracked.")
    else:
        print("Could not read second frame.")

cap.release()

```

Lucas-Kanade Sparse Optical Flow



Lucas-Kanade Optical Flow completed.
Tracked points: 85

```
In [36]: import cv2
import numpy as np
import matplotlib.pyplot as plt

video_path = "video.mp4"

cap = cv2.VideoCapture(video_path)

ret, prev = cap.read()
ret2, curr = cap.read()

if not ret or not ret2:
    print("Error: Cannot read video frames.")
else:
    prev_gray = cv2.cvtColor(prev, cv2.COLOR_BGR2GRAY)
    curr_gray = cv2.cvtColor(curr, cv2.COLOR_BGR2GRAY)

    # Farneback Dense Optical Flow
    flow = cv2.calcOpticalFlowFarneback(
        prev_gray,
        curr_gray,
        None,
        0.5,
        3,
        15,
        3,
        5,
        1.2,
        0
    )
```

```
# Magnitude and angle
magnitude, angle = cv2.cartToPolar(
    flow[..., 0],
    flow[..., 1]
)

# Create HSV visualization
hsv = np.zeros_like(prev)
hsv[..., 1] = 255

hsv[..., 0] = angle * 180 / np.pi / 2
hsv[..., 2] = cv2.normalize(
    magnitude,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)

output = cv2.cvtColor(
    hsv,
    cv2.COLOR_HSV2RGB
)

plt.figure(figsize=(10, 6))
plt.imshow(output)
plt.title("Farneback Dense Optical Flow")
plt.axis("off")
plt.show()

print("Farneback Dense Optical Flow completed.")

cap.release()
```

Farneback Dense Optical Flow



Farneback Dense Optical Flow completed.

```
In [38]: import cv2
import numpy as np
import matplotlib.pyplot as plt

video_path = "video.mp4"

cap = cv2.VideoCapture(video_path)

ret, first_frame = cap.read()

if not ret:
    print("Error: Cannot open video.")
else:

    prev_gray = cv2.cvtColor(first_frame, cv2.COLOR_BGR2GRAY)

    print("Farneback Dense Optical Flow initialized.")
```

Farneback Dense Optical Flow initialized.

```
In [39]: for i in range(20):

    ret, frame = cap.read()

    if not ret:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Farneback Dense Optical Flow
    flow = cv2.calcOpticalFlowFarneback(
        prev_gray,
        gray,
        None,
        0.5,
        3,
        15,
        3,
        5,
        1.2,
        0
    )

    # Convert flow to magnitude and angle
    magnitude, angle = cv2.cartToPolar(
        flow[..., 0],
        flow[..., 1]
    )

    # Create HSV image
    hsv = np.zeros_like(frame)

    # Hue represents direction
    hsv[..., 0] = angle * 180 / np.pi / 2
```

```
# Saturation
hsv[..., 1] = 255

# Value represents motion magnitude
hsv[..., 2] = cv2.normalize(
    magnitude,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)

# Convert HSV to BGR
flow_visual = cv2.cvtColor(
    hsv,
    cv2.COLOR_HSV2RGB
)

plt.figure(figsize=(10, 6))
plt.imshow(flow_visual)
plt.title("Farneback Dense Optical Flow")
plt.axis("off")
plt.show()

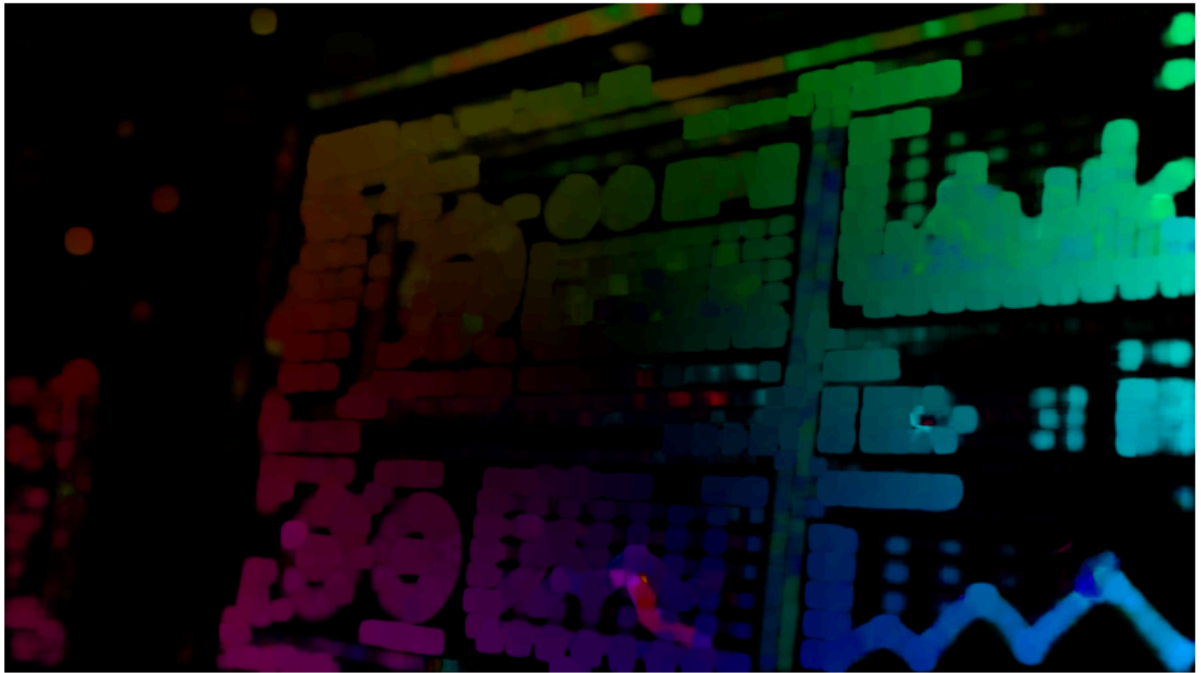
prev_gray = gray.copy()

cap.release()
```

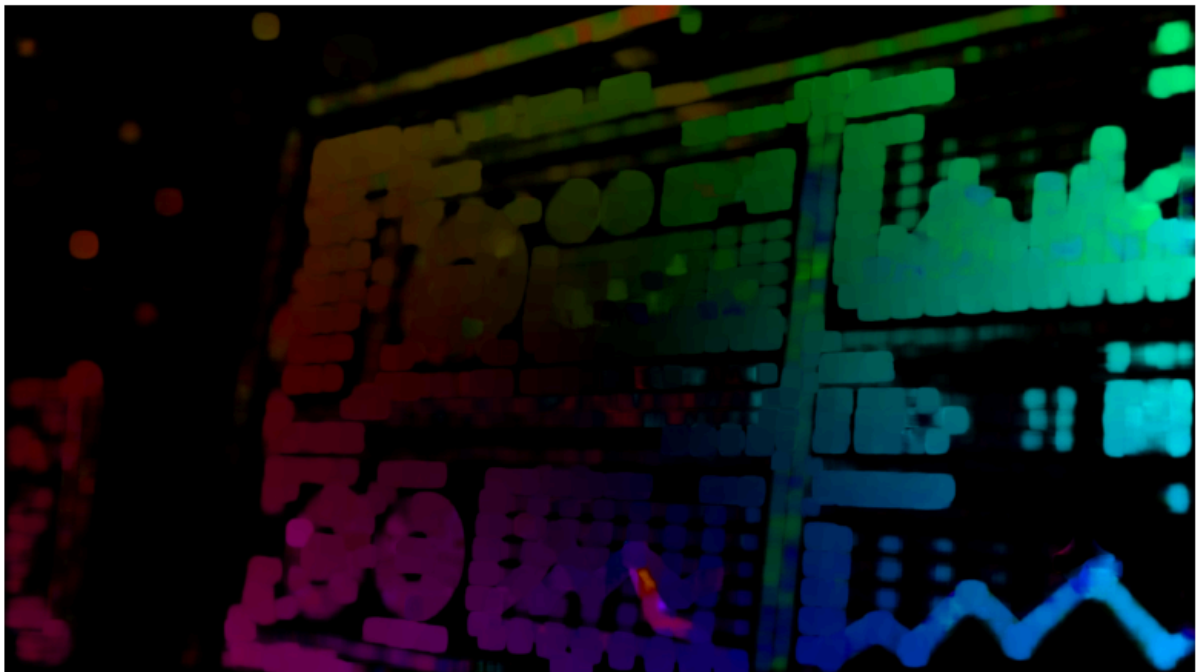
Farneback Dense Optical Flow



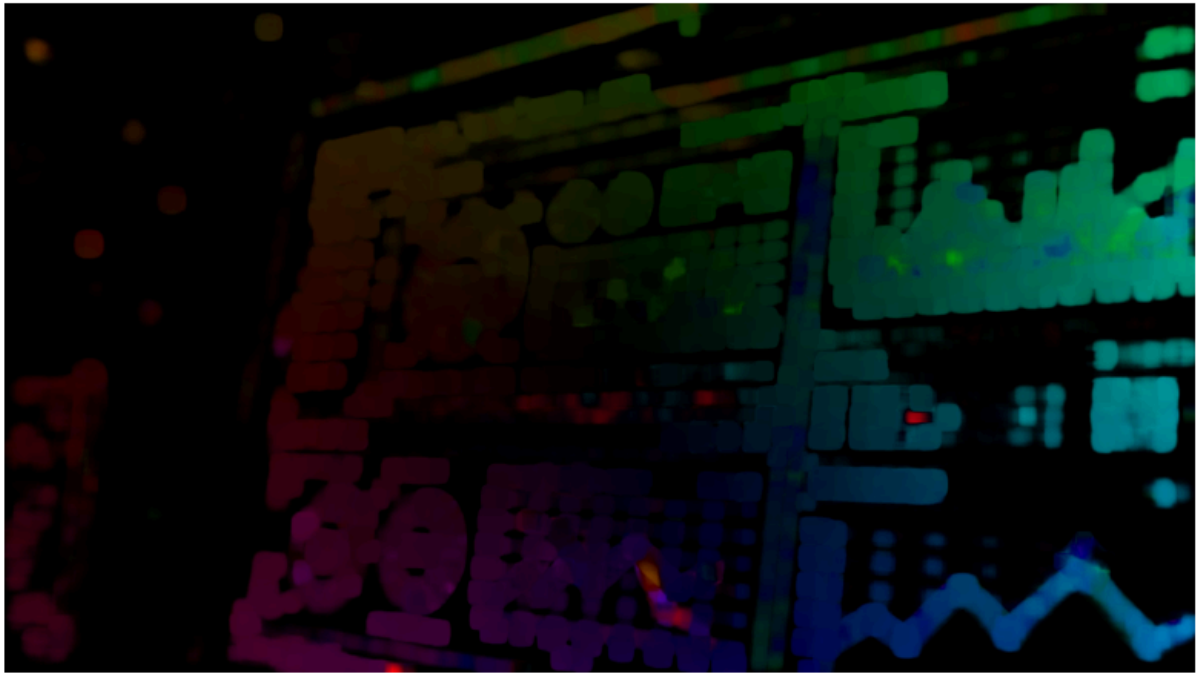
Farneback Dense Optical Flow



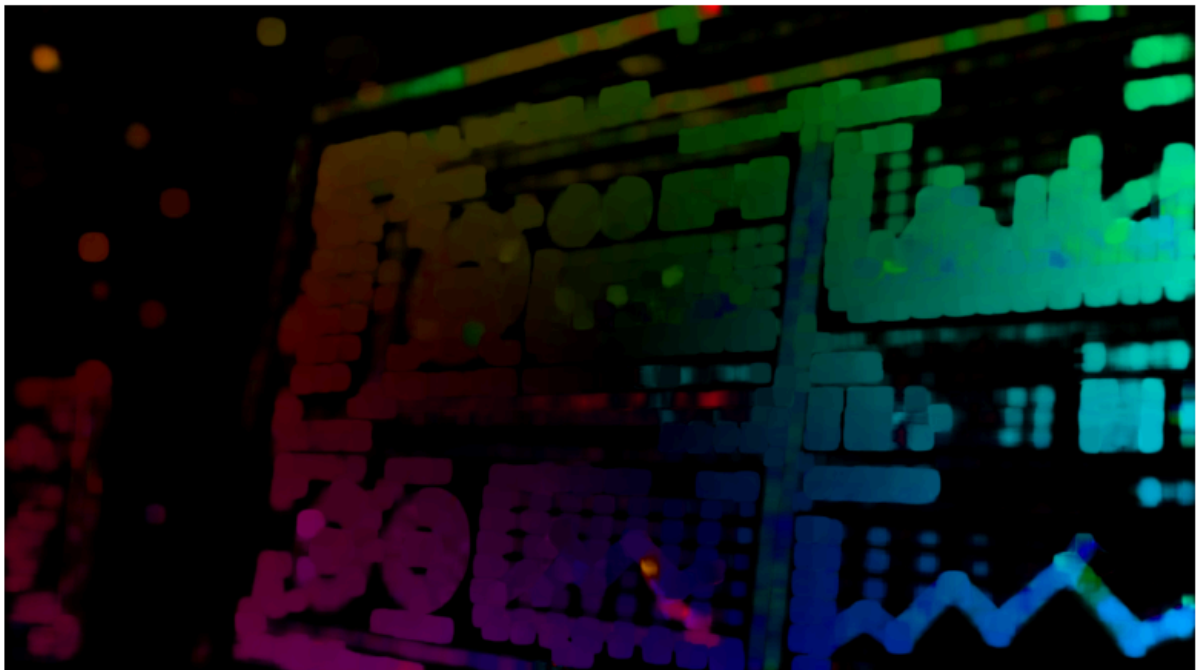
Farneback Dense Optical Flow



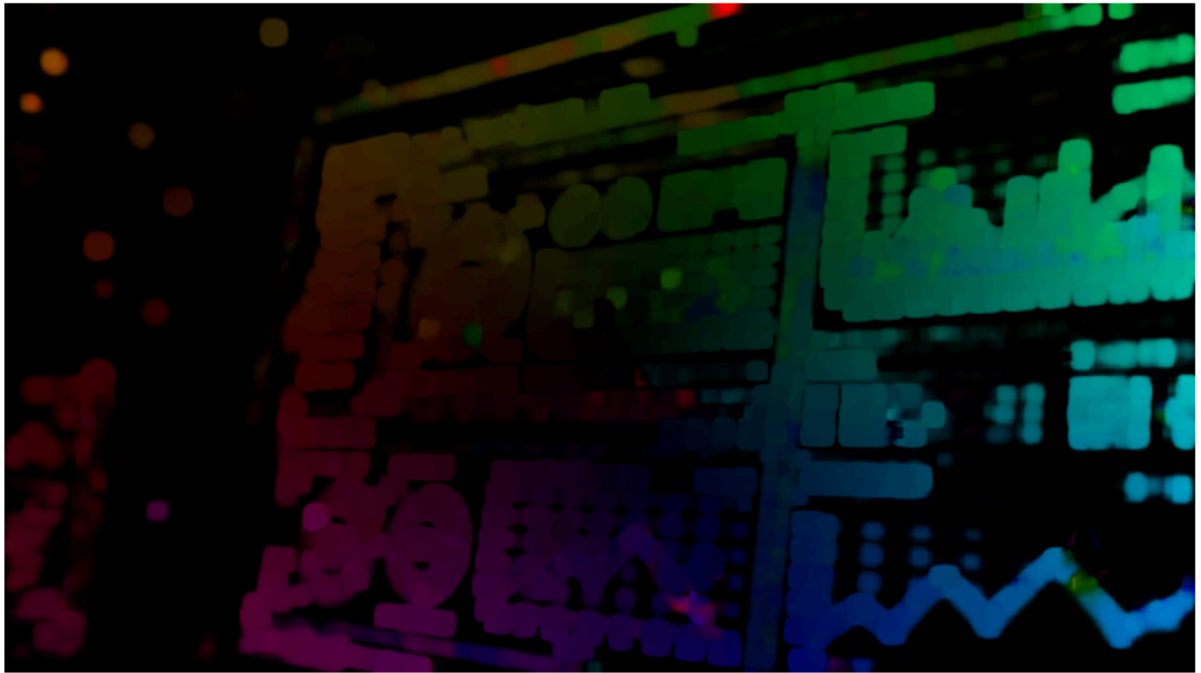
Farneback Dense Optical Flow



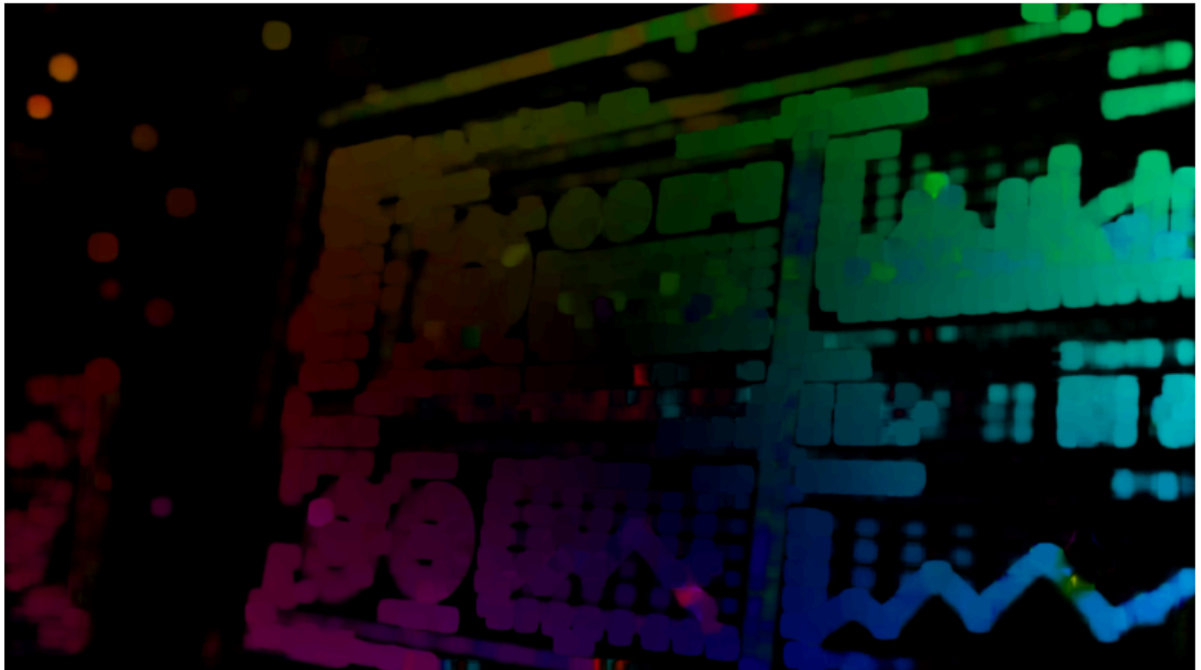
Farneback Dense Optical Flow



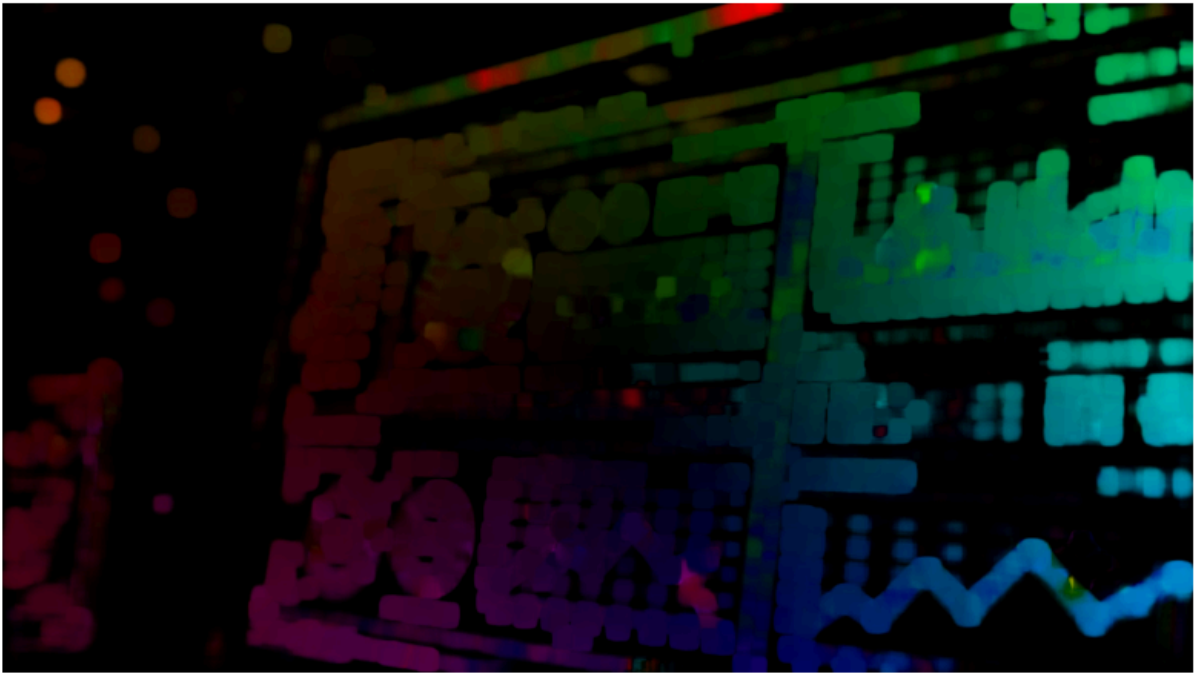
Farneback Dense Optical Flow



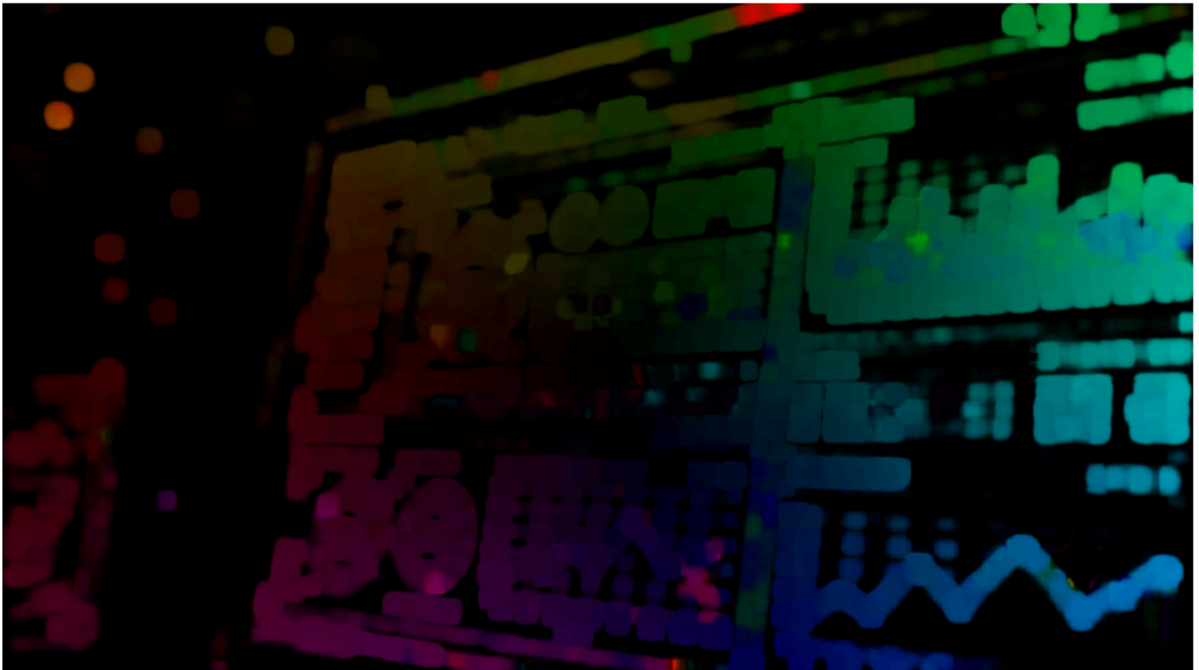
Farneback Dense Optical Flow



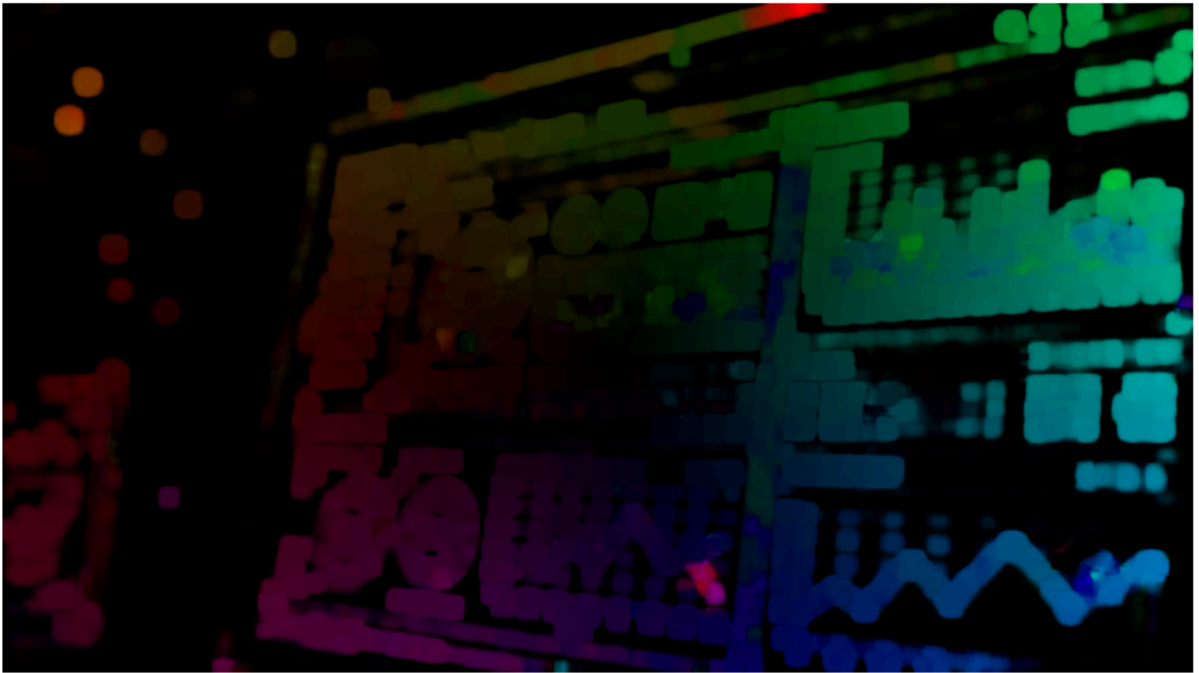
Farneback Dense Optical Flow



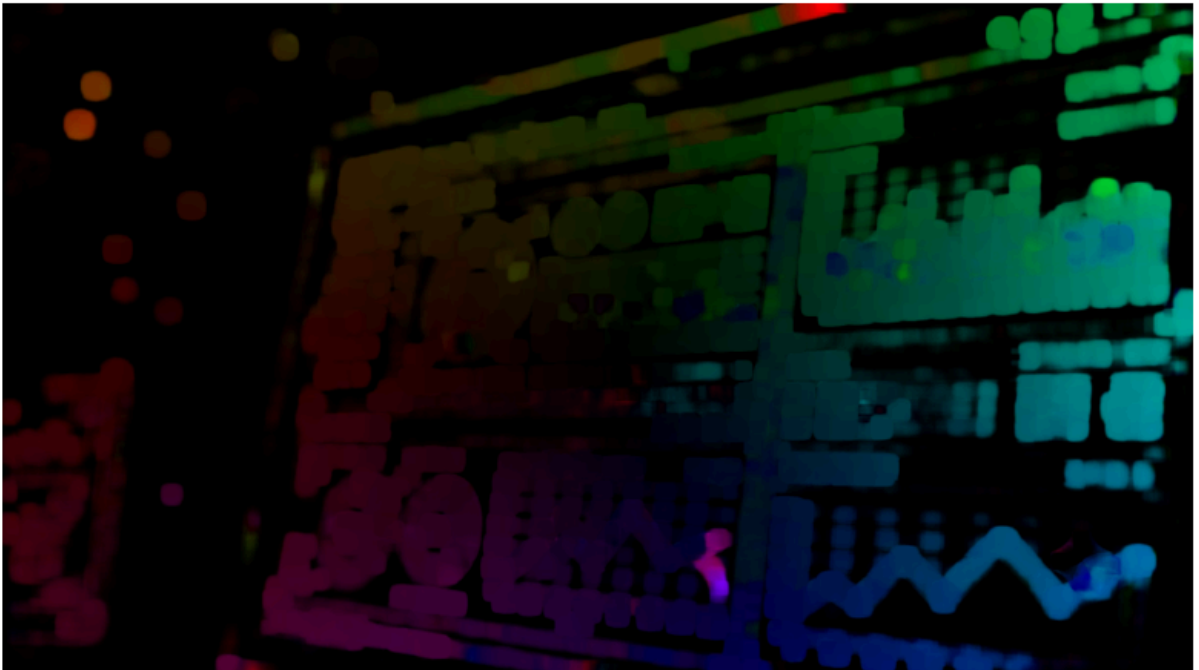
Farneback Dense Optical Flow



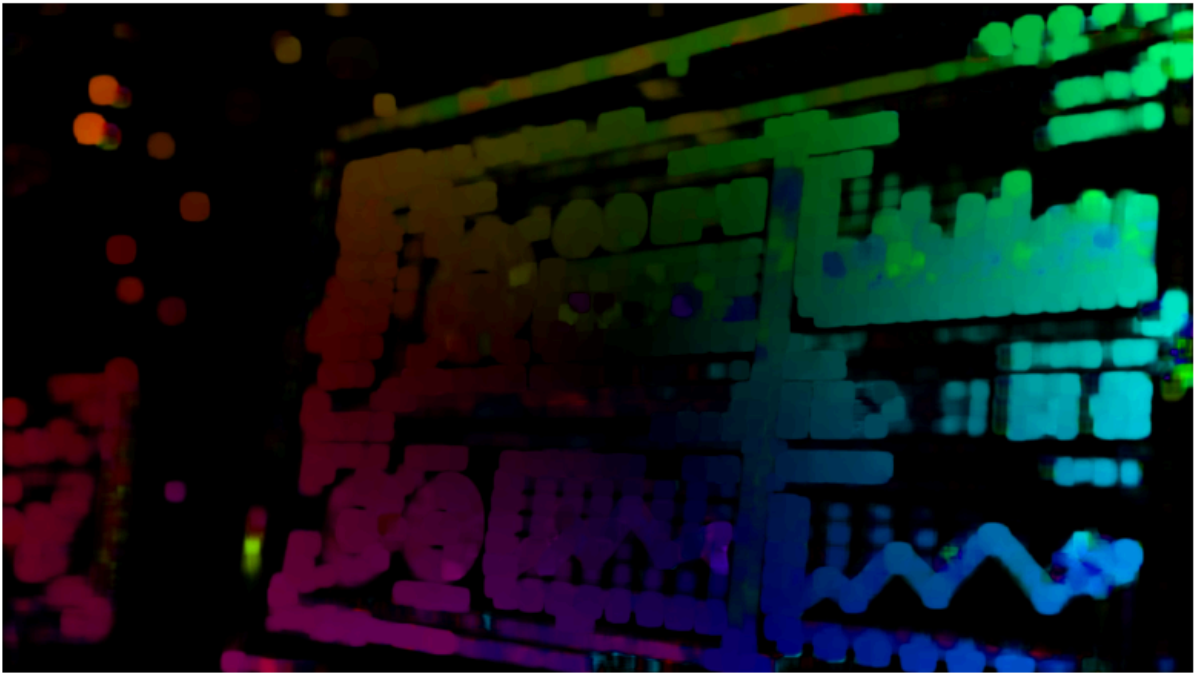
Farneback Dense Optical Flow



Farneback Dense Optical Flow



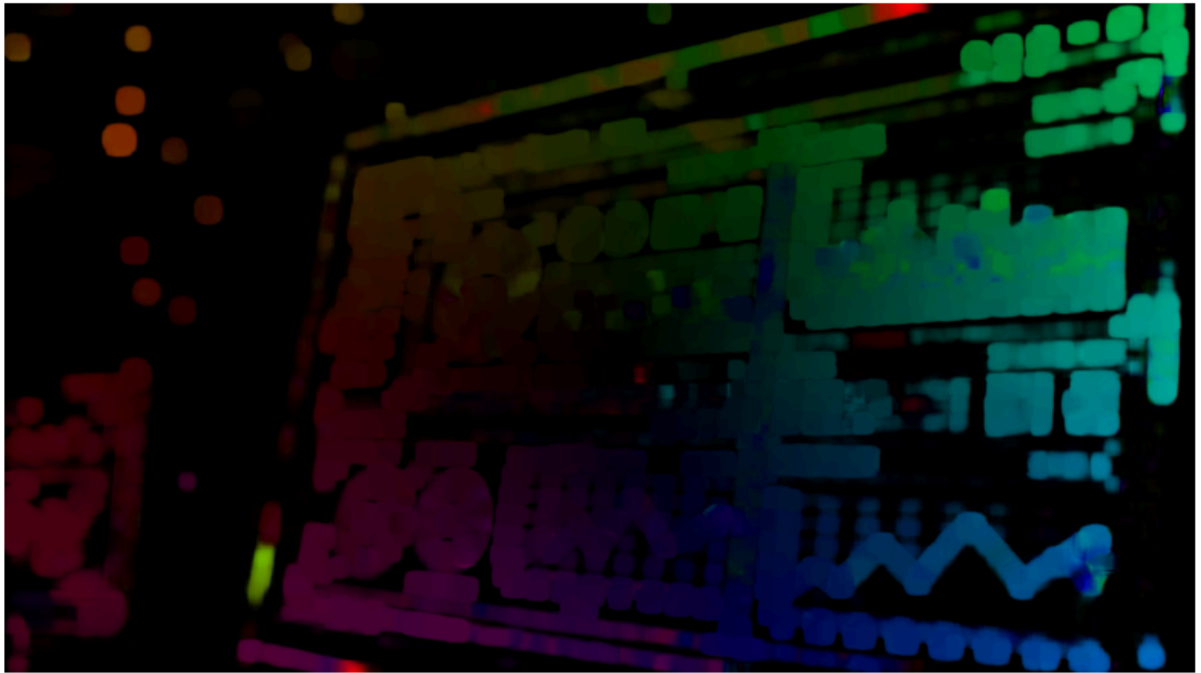
Farneback Dense Optical Flow



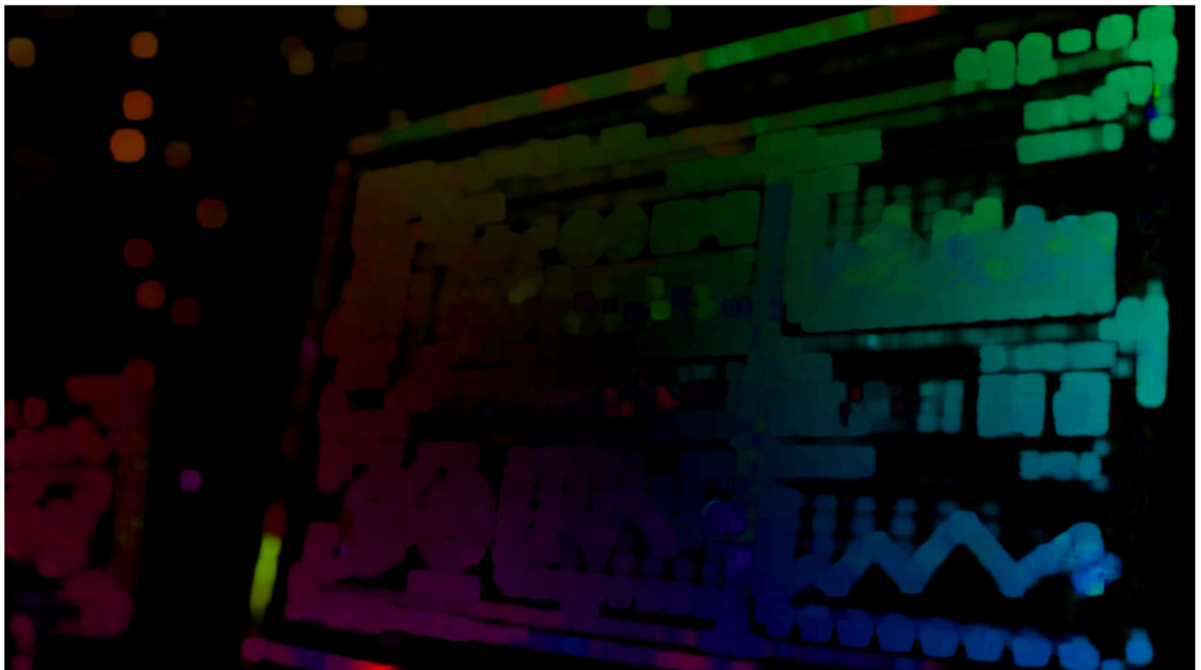
Farneback Dense Optical Flow



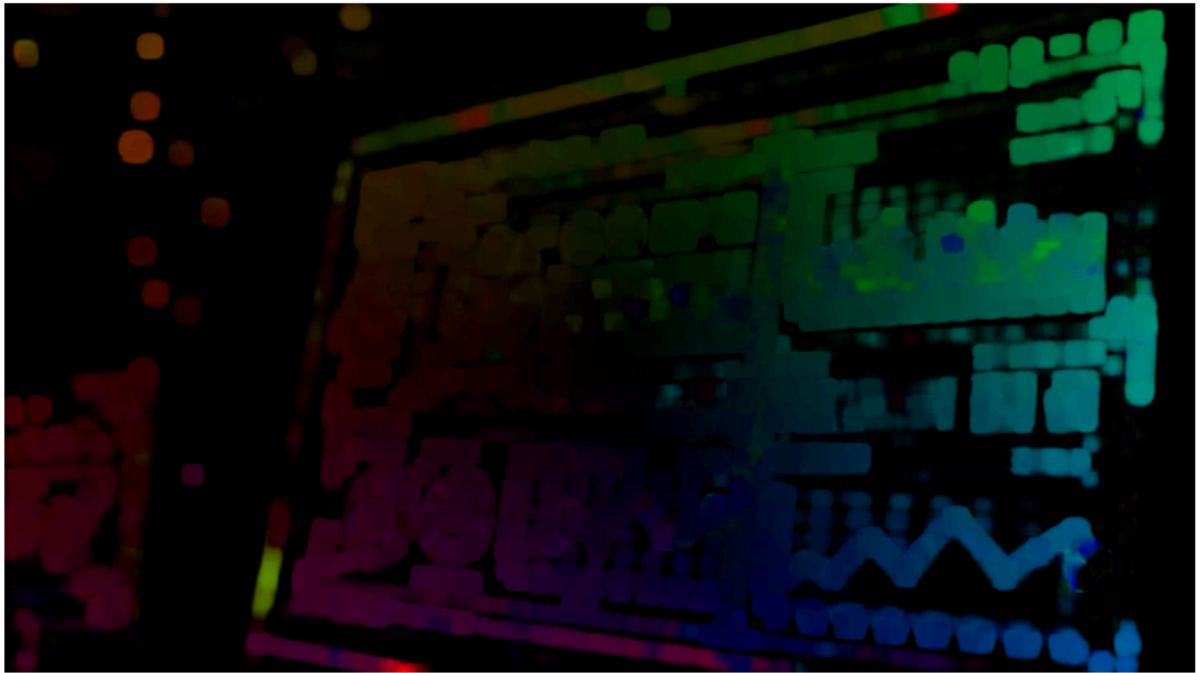
Farneback Dense Optical Flow



Farneback Dense Optical Flow



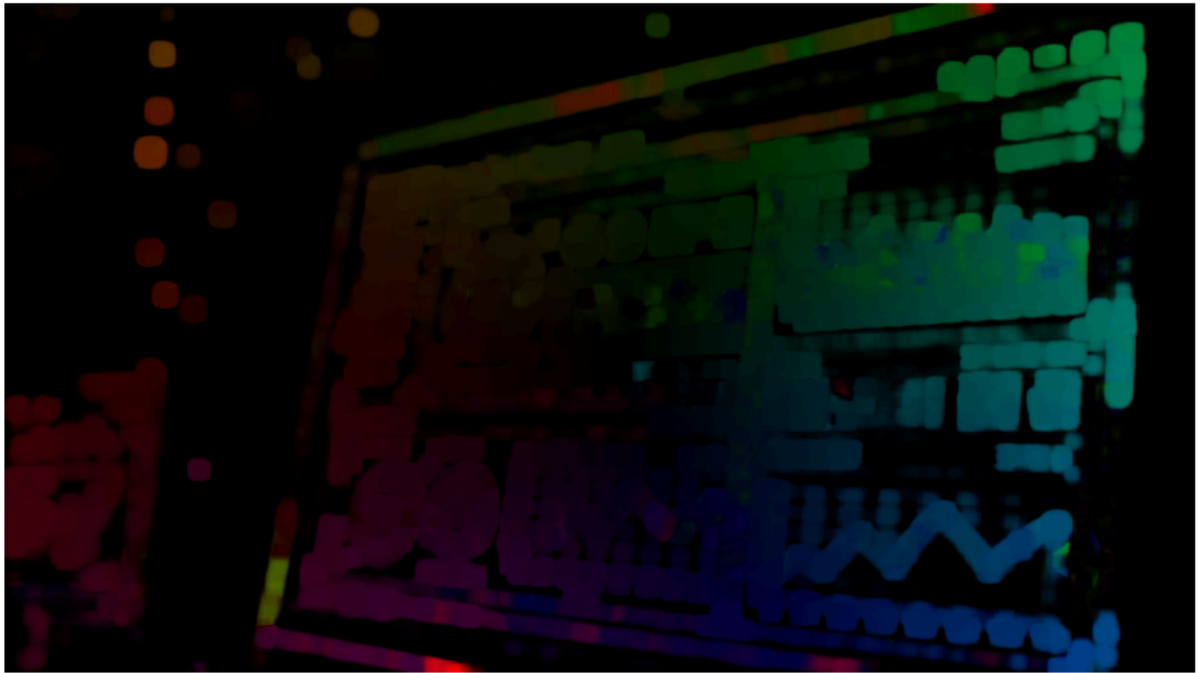
Farneback Dense Optical Flow



Farneback Dense Optical Flow



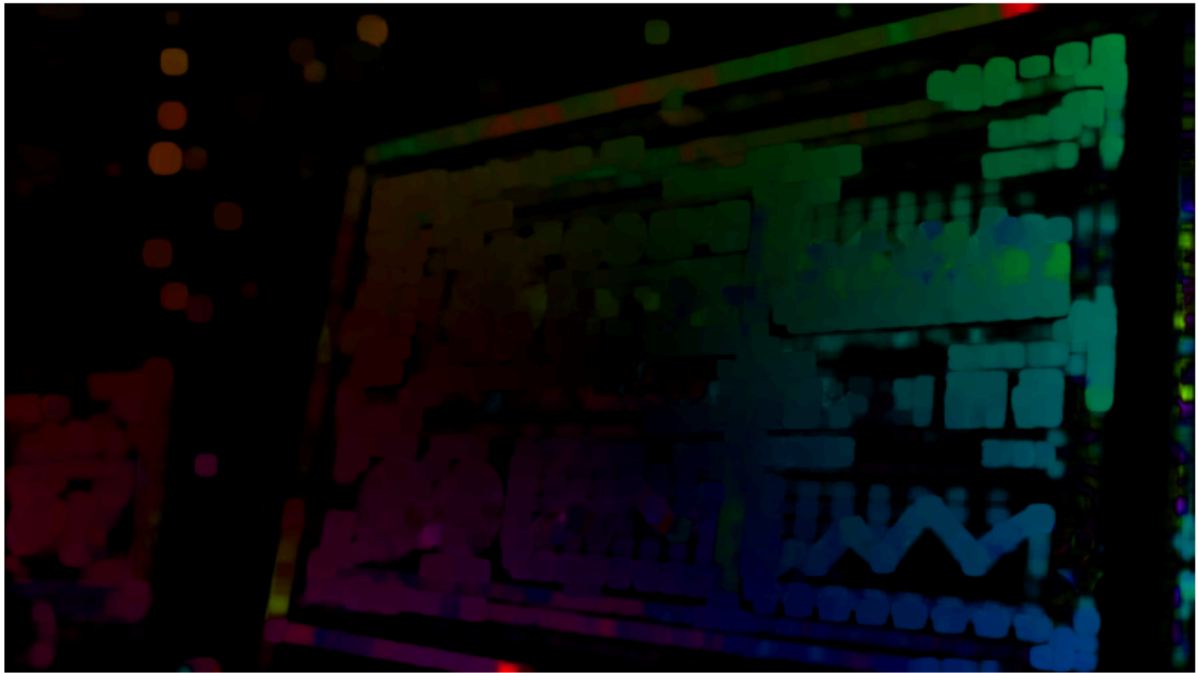
Farneback Dense Optical Flow



Farneback Dense Optical Flow



Farneback Dense Optical Flow



13. Understanding the Farneback Output

The output uses colors to represent motion.

Generally:

Hue/color → direction of movement
 Brightness/intensity → magnitude/speed of movement

Therefore, the Farneback method gives a complete motion field over the image.

14. Comparison Feature Lucas-Kanade Farneback Type Sparse Dense Tracks

Selected feature points Almost every pixel OpenCV function

`calcOpticalFlowPyrLK()` `calcOpticalFlowFarneback()` Computational cost Lower

Higher Output Motion vectors at points Motion field Speed Generally faster

Generally slower Useful for Feature/object tracking Complete motion analysis

Visualization Arrows/trajectories Color-coded flow

15. Step 9 — Observation

You can write this in your practical file:

Observations Lucas-Kanade successfully tracked selected feature points between consecutive frames. Motion vectors were represented using arrows and points. Farneback calculated dense optical flow over a large portion of the image. The direction and magnitude of motion were visualized using color coding. Lucas-Kanade

requires fewer computations because it tracks selected features. Farneback provides more complete motion information but requires greater computational resources. Changes in lighting, object speed, camera movement, and image quality can affect optical-flow estimation.

16. Result Result

The Lucas-Kanade Sparse Optical Flow and Farneback Dense Optical Flow algorithms were successfully implemented using Python and OpenCV.

Lucas-Kanade was used to track selected feature points, while Farneback was used to estimate motion over the image. The motion vectors and color-coded optical-flow patterns demonstrated how optical flow can be used for dynamic scene analysis.